

Phase 2 Implementation Roadmap

Overview

This roadmap breaks Phase 2 development into four manageable sub-phases, each deliverable in 1 week. Total estimated timeline: 4-5 weeks. The order prioritizes immediate business value (admin insights) while building foundation for engagement features (notifications).

Development Phases

Phase 2A: MVP Admin Dashboard	→ Week 1
Phase 2B: Element Notes	→ Week 2
Phase 2C: Wishlist Context	→ Week 3
Phase 2D: Notification Engine	→ Week 4-5

Total: 4-5 weeks

Phase 2A: MVP Admin Dashboard

Timeline: Week 1 (5 days)

Priority: HIGH - Immediate business value

Goals

- Give staff visibility into collection activity
- Identify restocking priorities based on actual demand
- Track user engagement and collection growth

Features

1. Admin Overview Dashboard

Route: `/app/admin`

Components:

- 4 key stats cards (total users, collections, avg/user, active users)
- Top 10 most collected elements table
- Top 10 most wanted elements table
- Recent activity feed (last 10 actions)

Data Sources:

- Aggregations on existing `CollectionItem` table
- No new database models needed

Effort: 2 days

2. User Collections List

Route: `/app/admin/users`

Components:

- Searchable table of all users
- Columns: Name, Email, Elements Owned, Completion %, Last Active
- Click row → user detail modal
- Export to CSV button

Features:

- Real-time search (debounced)
- Client-side sorting
- Pagination (15 per page)

Effort: 1.5 days

3. Demand Insights Panel

Route: `/app/admin/demand`

Components:

- Out-of-Stock Watchlist table
- High-Demand Items table
- Export buttons for restocking decisions

Data:

- Aggregate wishlist counts per element
- Stock status from Shopify API
- Days out of stock calculation

Effort: 1.5 days

Technical Tasks

Database

- **No schema changes needed** (uses existing models)
- Add indexes if needed:

`prisma`

`@@index([state, updatedAt]) on CollectionItem`

API Routes

```
GET /api/admin/stats
GET /api/admin/users?search=&page=
GET /api/admin/demand?sort=
POST /api/admin/export/users
POST /api/admin/export/demand
```

Access Control

- Middleware to check `User.isStaff = true`

- Redirect non-staff to `/app/cabinet` with error toast

UI Components

- `StatsCard.jsx` - Key metrics display
 - `DataTable.jsx` - Reusable sortable table
 - `ExportButton.jsx` - CSV generation
 - `UserDetailModal.jsx` - User info popup
-

Testing Requirements

Unit Tests

- ☐ Stats aggregation functions
- ☐ User search logic
- ☐ Demand calculation algorithm
- ☐ CSV export formatting

Integration Tests

- ☐ Admin routes return correct data
- ☐ Non-staff users blocked from access
- ☐ Export generates valid CSV

Manual QA

- ☐ Dashboard loads in <2s with 250+ users
 - ☐ Search results update in <500ms
 - ☐ Export downloads successfully
-

Success Criteria

- ☒ Staff can view all collection stats in one dashboard
 - ☒ Staff can find any user's collection in <30 seconds
 - ☒ Staff can export restocking list in 1 click
 - ☒ All admin pages load in <2 seconds
-

Deployment Checklist

- ☐ Add `isStaff` flag to admin user accounts
 - ☐ Test admin routes on staging
 - ☐ Train staff on new dashboard features
 - ☐ Document admin workflows
-

Phase 2B: Element Notes

Timeline: Week 2 (5 days)

Priority: MEDIUM - High user engagement value

Goals

- Let collectors add personal context to collection items
- Increase emotional connection to collection
- Enable private record-keeping

Features

1. Element Notes Database Model

New Model: `ElementNote`

Fields:

- acquisitionDate, source, pricePaid, condition, storageLocation, notes
- isPrivate flag
- 1:1 relationship with CollectionItem

Migration:

```
npx prisma migrate dev --name add_element_notes
```

Effort: 0.5 days

2. Notes Drawer UI

Component: `ElementNotesDrawer.jsx`

Layouts:

- Desktop: 400px slide-in drawer from right
- Mobile: Full-screen modal

Features:

- 7 form fields (all optional)
- Date picker for acquisition date
- Currency input for price
- Dropdown for condition
- Textarea for notes
- Privacy toggle

Effort: 2 days

3. Integration with Cabinet/Collection Views

Changes:

- Add “Add Notes” button to element cards
- Show  indicator when notes exist
- Click button → open notes drawer
- Close drawer → refresh card to show indicator

Files to Modify:

- `ElementCard.jsx`

- `CollectionGrid.jsx`
- `PeriodicTable.jsx`

Effort: 1 day

4. API Endpoints

```
GET /api/notes/:collectionItemId
POST /api/notes
PUT /api/notes/:noteId
DELETE /api/notes/:noteId
```

Validation:

- Zod schema for input validation
- Check user owns the collection item
- Validate date not in future
- Validate price range (0.01 - 999,999.99)

Effort: 1 day

5. Auto-Save & Loading States

Features:

- Loading spinner while fetching notes
- "Saving..." button state
- Success toast on save
- Error handling with retry

Effort: 0.5 days

Technical Tasks

Database

- Create `ElementNote` model
- Add relation to `CollectionItem`
- Create indexes on `userId` and `collectionItemId`

Components

- `ElementNotesDrawer.jsx` - Main drawer component
- `NotesForm.jsx` - Form fields
- `DatePicker.jsx` - Date selection
- `CurrencyInput.jsx` - Price input
- `ConditionSelector.jsx` - Condition dropdown

State Management

- Use React Hook Form for form state
- Optimistic updates (update UI, then save)

- Cache notes locally for 5 minutes
-

Testing Requirements

Unit Tests

- ☐ Form validation (date, price, text length)
- ☐ Currency formatting
- ☐ Date picker edge cases

Integration Tests

- ☐ Create note via API
- ☐ Update existing note
- ☐ Delete note
- ☐ Privacy flag respected

Manual QA

- ☐ Drawer opens smoothly on desktop
 - ☐ Modal works on mobile
 - ☐ All fields save correctly
 - ☐ Indicator appears after save
-

Success Criteria

- ☒ Users can add notes to any collection item in <30 seconds
 - ☒ Notes persist across sessions
 - ☒ Drawer opens/closes without lag
 - ☒ All fields save correctly with validation
-

Deployment Checklist

- ☐ Run database migration
 - ☐ Test notes drawer on staging
 - ☐ Verify mobile layout
 - ☐ Add notes feature to user onboarding
-

Phase 2C: Wishlist Context

Timeline: Week 3 (5 days)

Priority: MEDIUM - Improves user prioritization

Goals

- Help collectors prioritize wishlist purchases
- Add semantic meaning to wishlist items
- Enable filtering by intent (Core, Exploration, Aspirational, Upgrade)

Features

1. Wishlist Context Detection Logic

Algorithm: Auto-assign context based on:

- User's collection state (completion %)
- Element price and rarity
- Existing ownership (for upgrades)

Context Types:

- CORE: <50% category completion, common elements, <\$50
- EXPLORATION: 50-90% category completion, mid-range price
- ASPIRATIONAL: >\$100 OR <20% global ownership
- UPGRADE_TARGET: User owns element in different format

Implementation:

- `lib/wishlist-context.js` - Detection algorithm
- Called when user adds item to wishlist
- Stored in `CollectionItem.contextLabel`

Effort: 1.5 days

2. Database Schema Update

Changes to CollectionItem:

```
contextLabel      WishlistContext?
contextOverridden Boolean           @default(false)
contextReason     String?
```

New Enum:

```
enum WishlistContext {
  CORE
  EXPLORATION
  ASPIRATIONAL
  UPGRADE_TARGET
}
```

Migration:

```
npx prisma migrate dev --name add_wishlist_context
```

Effort: 0.5 days

3. Enhanced Wishlist UI

Changes to `/app/wishlist` :

Features:

- Group wishlist items by context

- Collapsible sections per context
- Visual badges with distinct colors
- Filter chips (All, Core, Exploration, etc.)
- “Edit Label” button on each item

Components:

- `WishlistContextGroup.jsx` - Section wrapper
- `ContextBadge.jsx` - Color-coded badge
- `ContextFilterBar.jsx` - Filter chips
- `ContextSelectionModal.jsx` - Override modal

Effort: 2 days

4. Context Assignment Modal

Trigger: Click “Edit Label” on wishlist item

Features:

- Radio button selection of context
- Show recommended context with explanation
- Gray out invalid options (e.g., Upgrade if not owned)
- Save and update grouping

Effort: 1 day

5. Context Analytics for Admin

Add to Admin Demand Insights:

- Context distribution breakdown
- Most wanted by context (top 5 Core, top 5 Aspirational, etc.)

Queries:

```
await prisma.collectionItem.groupBy({
  by: ['contextLabel'],
  where: { state: 'WISHLIST' },
  _count: { id: true }
});
```

Effort: 0.5 days (piggyback on existing admin dashboard)

Technical Tasks

Database

- Add 3 new fields to `CollectionItem`
- Create `WishlistContext` enum
- Add index on `contextLabel`

Logic

- `detectWishlistContext()` - Main algorithm
- `calculateRelevanceScore()` - Scoring helper
- `getGlobalOwnership()` - Stats query
- `getCategoryCompletion()` - User progress

UI Components

- Update wishlist route with grouping
 - Add filter bar
 - Create context badge component
 - Build override modal
-

Testing Requirements

Unit Tests

- ☐ Context detection algorithm accuracy
- ☐ Edge cases (user owns all formats, price edge cases)
- ☐ Override logic

Integration Tests

- ☐ Context saved on wishlist add
- ☐ Context persists after page refresh
- ☐ Manual override prevents auto-updates
- ☐ Filter works correctly

Manual QA

- ☐ Wishlist groups display correctly
 - ☐ Context badges show right colors
 - ☐ Filter chips work smoothly
 - ☐ Override modal saves correctly
-

Success Criteria

- ☒ 80%+ of wishlist items auto-labeled with correct context
 - ☒ Users can filter wishlist by context in 1 click
 - ☒ Manual overrides persist across sessions
 - ☒ Admin can see context distribution
-

Deployment Checklist

- ☐ Run database migration
- ☐ Backfill context labels for existing wishlist items
- ☐ Test detection algorithm accuracy
- ☐ Update user docs with context explanations

Phase 2D: Notification Engine

Timeline: Week 4-5 (7 days)

Priority: HIGH - Drives engagement and sales

Goals

- Re-engage collectors with timely, personalized alerts
- Notify about wishlist restocks (drives sales)
- Celebrate milestones (retention)
- Surface high-relevance products (discovery)

Features

1. Notification Database Models

New Models:

- NotificationQueue - All notifications
- UserNotificationPreferences - User settings
- StockAlert - Restock tracking

Migration:

```
npx prisma migrate dev --name add_notifications
npx prisma db seed # Create default preferences for existing users
```

Effort: 1 day

2. Notification Trigger Detection

5 Trigger Types:

1. **Completion Unlock** (1 element away from milestone)
 - Runs on collection update
 - Checks if adding any wishlist item would cross threshold
2. **Near-Completion** (90%+ category completion)
 - Runs on collection update
 - One-time per category
3. **Wishlist Restock** (item back in stock)
 - Runs on Shopify product update webhook
 - Finds all users with item on wishlist
4. **High-Relevance Arrival** (new product matching interests)
 - Runs on new product added
 - Calculates relevance score for all users
5. **Milestone Celebration** (milestone reached)
 - Runs on collection update
 - Immediate notification

Implementation:

- `lib/notifications/triggers/` - 5 trigger functions
- `lib/notifications/queue.js` - Queue management
- `lib/notifications/frequency.js` - Frequency limiting

Effort: 2 days

3. Notification Queue Processor

Background Job:

- Runs every 1 minute (cron or setInterval)
- Fetches pending notifications
- Checks user preferences
- Delivers via in-app + email
- Updates status to SENT or FAILED

Implementation:

- `lib/notifications/processor.js` - Main processor
- `lib/notifications/delivery/in-app.js` - In-app delivery
- `lib/notifications/delivery/email.js` - Email delivery

Email Service: SendGrid or Nodemailer SMTP

Effort: 1.5 days

4. In-App Notification Center

Route: `/app/notifications`

Features:

- List of all notifications (newest first)
- Filter: All / Unread Only
- Mark as read
- Click notification → navigate to relevant page (cabinet, shop, etc.)
- Pagination (20 per page)

Components:

- `NotificationCenter.jsx` - Main page
- `NotificationList.jsx` - List view
- `NotificationCard.jsx` - Single notification
- `NotificationBell.jsx` - Top nav bell icon with badge

Effort: 1.5 days

5. Notification Preferences UI

Route: `/app/notifications/preferences`

Features:

- Toggle each notification type (5 types × 2 channels = 10 toggles)

- Email frequency limit slider (1-10 per week)
- “Mute all until” date picker
- Save button

Component: `NotificationPreferences.jsx`

Effort: 1 day

6. Email Templates

5 Email Templates (HTML):

- Completion Unlock
- Near-Completion
- Wishlist Restock
- High-Relevance Arrival
- Milestone Celebration

Features:

- Responsive HTML
- Luciteria branding (colors, logo)
- CTA buttons
- Unsubscribe link → preferences page

Effort: 1 day (use template library like MJML)

Technical Tasks

Database

- Create 3 new models (NotificationQueue, UserNotificationPreferences, StockAlert)
- Seed default preferences for all users
- Indexes on status, scheduledFor, userId

Background Processing

- Set up cron job or Node.js interval
- Error handling and retry logic
- Logging for debugging

Email Integration

- SendGrid API setup
- Email template compilation
- Tracking open/click rates (optional)

API Routes

```
GET  /api/notifications?filter=&page=
POST /api/notifications/:id/read
GET  /api/notifications/preferences
PUT  /api/notifications/preferences
POST /api/notifications/test/:type # Admin tool to test triggers
```

Webhooks (if using Shopify webhooks)

```
POST /api/webhooks/product-update # Detect restocks
POST /api/webhooks/product-create # Detect new arrivals
```

Testing Requirements

Unit Tests

- ☐ Each trigger detection function
- ☐ Frequency limiting logic
- ☐ Context scoring algorithm
- ☐ Email template rendering

Integration Tests

- ☐ Queue notification end-to-end
- ☐ Processor picks up and delivers
- ☐ User preferences respected
- ☐ Email sends successfully
- ☐ In-app notification appears

Manual QA

- ☐ Trigger each notification type manually
- ☐ Verify email delivery
- ☐ Check in-app notification appears
- ☐ Test frequency limiting
- ☐ Test opt-out (no emails sent)

Success Criteria

- ☒ All 5 notification types triggering correctly
- ☒ Emails delivered within 5 minutes of trigger
- ☒ In-app notifications appear immediately
- ☒ User preferences fully respected
- ☒ No duplicate notifications sent
- ☒ 80%+ email open rate (industry avg: 20-30%)

Deployment Checklist

- ☐ Run database migrations
- ☐ Seed notification preferences for all users
- ☐ Set up SendGrid account and API key
- ☐ Configure email templates
- ☐ Start background processor (cron or PM2)
- ☐ Test each notification type on staging

- [] Monitor processor logs for first 48 hours
 - [] Set up email deliverability monitoring
-

Phase 2 Complete: Summary

Total Timeline

- **Phase 2A:** Week 1 (Admin Dashboard)
- **Phase 2B:** Week 2 (Element Notes)
- **Phase 2C:** Week 3 (Wishlist Context)
- **Phase 2D:** Week 4-5 (Notifications)

Total: 4-5 weeks

Delivered Features

Admin Tools

- Collection intelligence dashboard
- User management and search
- Demand insights for restocking

User Engagement

- Personal element notes
- Wishlist context labels
- 5-channel notification system
- In-app notification center

Data & Intelligence

- Context-aware wishlist
 - Smart product relevance scoring
 - Engagement analytics
-

Technical Deliverables

New Database Models

- NotificationQueue
- UserNotificationPreferences
- ElementNote
- StockAlert
- AnalyticsSnapshot

New API Routes

- 3 admin routes
- 4 notification routes
- 4 notes routes
- 2 webhook routes

New UI Components

- 15+ React components
 - 3 new routes
 - 1 drawer component
 - 5 email templates
-

Success Metrics (3 Months Post-Launch)

For Users

- **Engagement:** 40% of users add at least one element note
- **Wishlist Usage:** 70% of users label wishlist items with context
- **Notification Opt-In:** 80%+ keep notifications enabled
- **Email Open Rate:** 50%+ (significantly above industry avg)

For Business

- **Conversion:** 15% lift in wishlist-to-purchase conversion
- **Retention:** 10% increase in 30-day active users
- **Restocking:** 20% faster restock decisions using demand insights
- **Support:** 30% reduction in “when will X restock?” inquiries

For Staff

- **Admin Efficiency:** Find user collections in <30 seconds (vs. manual Shopify queries)
 - **Data-Driven:** Restock decisions based on actual demand, not guesses
 - **Visibility:** Real-time view of collection activity
-

Post-Phase 2: Maintenance & Monitoring

Week 6-8: Stabilization Period

Monitor

- Notification delivery rates
- Email bounce/spam rates
- Background processor performance
- Admin dashboard load times
- User feedback on new features

Optimize

- Tune notification frequency limits based on user behavior
- Refine context detection algorithm accuracy
- Improve admin query performance if slow
- A/B test email templates for open rates

Bug Fixes

- Address any edge cases discovered
- Fix mobile UI issues
- Handle error states gracefully

Ongoing Tasks

Weekly

- Review notification analytics (open rates, clicks)
- Check for failed notifications in queue
- Monitor email deliverability scores

Monthly

- Analyze context distribution trends
- Review most-wanted elements list
- Assess notification effectiveness by type
- User surveys on new features

Quarterly

- Major feature enhancements (Phase 3 planning)
 - Performance optimization
 - Security audits
-

Phase 3 Preview (Future)

Potential Features (Not in Phase 2)

- **Social Features:** Share collections, public profiles
 - **Advanced Admin:** Bulk operations, automated campaigns
 - **Gamification:** Badges, leaderboards, challenges
 - **Mobile App:** Native iOS/Android apps
 - **Advanced Analytics:** Predictive restocking, demand forecasting
 - **Community:** Forums, collector meetups, trades
-

Risk Management

Potential Risks & Mitigation

1. Notification Spam

Risk: Users receive too many emails, opt out

Mitigation: Strict frequency limits, smart relevance scoring, easy opt-out

2. Performance Issues

Risk: Admin dashboard slow with large dataset

Mitigation: Query optimization, caching, pagination, indexes

3. Email Deliverability

Risk: Emails marked as spam

Mitigation: Use reputable service (SendGrid), proper SPF/DKIM setup, monitor bounce rates

4. Context Detection Accuracy

Risk: Auto-labels don't match user intent

Mitigation: Allow manual override, refine algorithm based on feedback

5. Scope Creep

Risk: Feature requests delay timeline

Mitigation: Strict MVP focus, defer enhancements to Phase 3

Resource Requirements

Development Team

- **1 Full-Stack Developer:** All phases
- **1 Designer (Part-Time):** UI review, email templates
- **1 QA Tester (Part-Time):** Week 5 (full phase testing)

Infrastructure

- **Database:** PostgreSQL (existing)
- **Email Service:** SendGrid (~\$15-50/month for 10K emails)
- **Background Jobs:** Node.js cron or PM2
- **Monitoring:** Sentry or similar (error tracking)

Budget Estimate

- **Development:** 4-5 weeks × developer rate
- **Infrastructure:** \$50-100/month (email service, monitoring)
- **Testing:** 1 week QA time
- **Total:** ~\$15K-25K (depending on rates)

Implementation Best Practices

Code Quality

- ESLint + Prettier for consistency
- TypeScript for type safety (optional but recommended)
- Unit test coverage >80%
- Code reviews for all PRs

Git Workflow

- Feature branches for each sub-phase
- PR to `develop` branch
- Merge to `main` after QA pass
- Tag releases (v2.0, v2.1, etc.)

Deployment Strategy

- Deploy each sub-phase to staging first
- Staff testing period (2-3 days)

- Deploy to production after approval
- Monitor for 24 hours post-deploy

Documentation

- Update README with new features
 - API documentation for new endpoints
 - User-facing docs for notification settings
 - Admin training materials
-

Final Checklist

Phase 2A Complete

- ☐ Admin dashboard deployed
- ☐ All 3 admin routes functional
- ☐ Export buttons working
- ☐ Staff trained on dashboard

Phase 2B Complete

- ☐ Element notes model in production
- ☐ Notes drawer functional on all devices
- ☐ All form fields saving correctly
- ☐ Notes indicator showing on cards

Phase 2C Complete

- ☐ Wishlist context detection live
- ☐ Context labels showing on wishlist
- ☐ Filter chips working
- ☐ Manual override functional
- ☐ Admin context analytics live

Phase 2D Complete

- ☐ All 5 notification triggers active
- ☐ Notification queue processor running
- ☐ In-app notification center deployed
- ☐ Email delivery working
- ☐ Preferences page functional
- ☐ Monitoring in place

Phase 2 Complete

- ☐ All features tested end-to-end
 - ☐ Performance targets met
 - ☐ User documentation published
 - ☐ Monitoring dashboards set up
 - ☐ Feedback collection mechanism in place
-

Conclusion

Phase 2 transforms the Luciteria Collector Cabinet from a tracking tool into an engagement platform. By adding admin intelligence, personal context, smart prioritization, and timely notifications, we create a system that actively helps collectors grow their collections while giving staff the data they need to optimize inventory.

Key Principle: Ship iteratively. Each sub-phase delivers standalone value while building toward the complete vision.

Success Definition: Users check their wishlist weekly (up from monthly), staff make restock decisions in minutes (down from hours), and notifications drive measurable conversion lift.

Let's build it! 🚀